

openagent-logger

Capture layer for the OpenAgent system. Receives signed events from `openagent-api` and stores them append-only in monthly-partitioned PostgreSQL tables.

Version	0.1.0
Port	8003
Base image	python:3.11-slim
Database	PostgreSQL 13+
Schema	<code>openagent_logger</code>
Status	working — pre-production

Overview

`openagent-logger` is the capture layer of the OpenAgent system. It owns the **wire endpoint** that `openagent-api` calls to record three classes of event:

- **ops_events** — operational telemetry (request_received, auth_failure, upstream_error, stream_complete, etc.). Short retention (default 90 days).
- **conversation_captures** — full `/chat` content (input, output, token counts, model used, latency). Medium retention (default 180 days). Captures arrive from any model configured in the BYOC infrastructure layer, distinguished by the `model_used` column.
- **audit_events** — security-relevant actions (key_rotation, admin_endpoint_hit, retention_job_run). Long retention (default ~7 years).

What `openagent-logger` **deliberately does not do**:

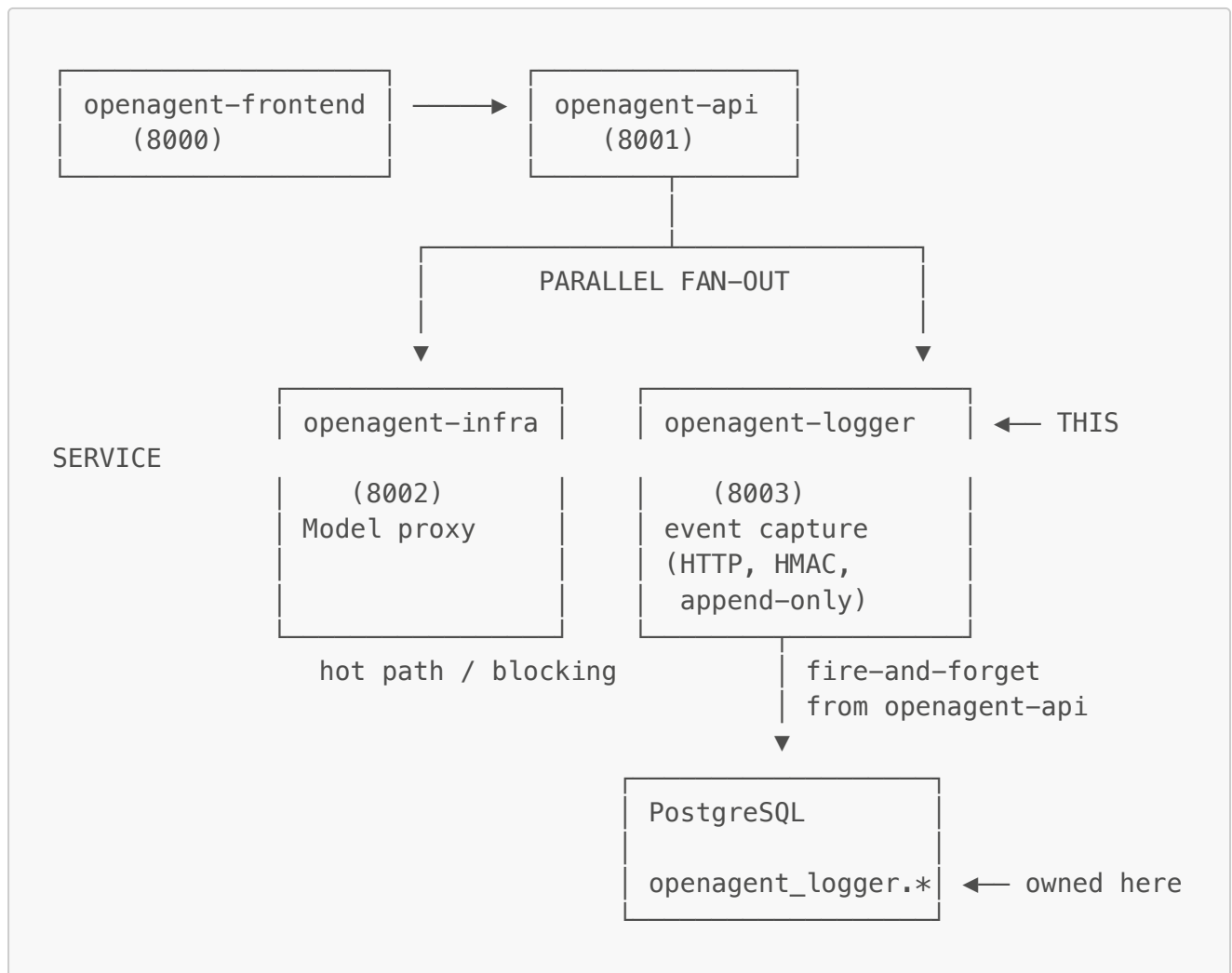
- No PII stripping.
- No quality scoring or training-data classification.
- No session lifecycle management.
- No user identity validation.
- No rate limiting → defer to a reverse proxy.

This separation keeps the capture path narrow, fast, and easy to reason about: validate the envelope, verify the HMAC, write the row, return.

A note on the event envelope. Every event carries an optional, nullable `session_id` and `user_id`. They are caller-supplied correlation fields — the logger accepts them, stores them, and never validates them. In the reference stack they arrive `null`; they exist so a caller that *does* track sessions or users can correlate events without a schema change. The logger has no opinion about either field beyond storing it.

Architecture

`openagent-logger` sits **in parallel with openagent-infra**, not in series. `openagent-api` calls both as siblings: `openagent-infra` on the hot path of `/chat` (must succeed for the user to get a response), and `openagent-logger` as fire-and-forget capture (must not block the response). The two services have no knowledge of each other — they share only `openagent-api` as their caller.



Why this matters for openagent-logger. Because it's parallel, not downstream of `openagent-infra`, `openagent-logger`:

- Never sees the model response stream directly. `openagent-api` assembles the full response from the BYOC provider's SSE chunks, then submits the finished pair as a single `conversation_capture`.
- Can be down without affecting `/chat` latency. `openagent-api` emits events fire-and-forget; events submitted while `openagent-logger` is down are **lost** (there is no emitter-side outbox — see Known limitations).
- Sees traffic equally from any configured model — whichever compute worker handled the `/chat` request, `openagent-api` emits a capture with `model_used` set accordingly. The schema is model-agnostic; downstream filtering happens at query time.

Quick example

Send an ops_event (after `cp .env.example .env` and `docker compose up`):

```
# These values must come from the SAME .env the service is using
export LOGGER_API_KEY="$(grep ^LOGGER_API_KEY .env | cut -d= -f2)"
export LOGGER_HMAC_SECRET="$(grep ^LOGGER_HMAC_SECRET .env | cut -d= -f2)"

# Build a minimal event and submit it
python - <<'PY'
import json, hmac, hashlib, os, uuid
from datetime import datetime, timezone
import httpx

request_id = str(uuid.uuid4())
ts = datetime.now(timezone.utc).isoformat()
payload = {"action": "request_received", "outcome": "success",
"details": {}}

canonical_payload = json.dumps(payload, sort_keys=True, separators=(",",
":"), default=str)
payload_hash = hashlib.sha256(canonical_payload.encode()).hexdigest()
canonical_string = f"{{request_id}}|{{ts}}|ops_event|{{payload_hash}}"
sig = hmac.new(
    os.environ["LOGGER_HMAC_SECRET"].encode(),
    canonical_string.encode(),
    hashlib.sha256,
).hexdigest()

body = {
    "event_type": "ops_event",
    "request_id": request_id,
    "source_service": "test-client",
    "client_timestamp": ts,
    "session_id": None,    # optional, nullable, caller-supplied; not
validated
    "user_id": None,      # optional, nullable, caller-supplied; not
validated
    "hmac_signature": sig,
    "payload": payload,
}

r = httpx.post(
    "http://localhost:8003/events",
    json=body,
    headers={"X-API-Key": os.environ["LOGGER_API_KEY"]},
)
print(r.status_code, r.json())
PY
```

A successful call returns `201 Created` with the assigned `event_id`.

API endpoints

A brief summary:

Method	Path	Auth	Purpose
POST	/events	X-API-Key + HMAC	Capture a signed event
GET	/health	X-API-Key	Service readiness probe
GET	/stats	X-API-Key	Row counts and timestamp bounds
GET	/	(none)	Service identification only

/health and /stats are intentionally authenticated — operational state at this boundary is internal information and not appropriate for an unauthenticated probe.

Security model

openagent-logger uses **two independent secrets**, each protecting against a different threat. Both must pass for a write to land.

Layer 1: Transport — `LOGGER_API_KEY`

Every inbound request must carry the header:

```
X-API-Key: <value of LOGGER_API_KEY>
```

Validated at the door using `hmac.compare_digest` (constant-time, no timing-attack leakage). Missing or wrong key → HTTP 401.

This protects against **any** unauthorised caller reaching the service. It is the same per-boundary compartmentalized pattern `openagent-api` uses with `OPENAGENT_API_KEY` and `INFRA_API_KEY`.

Layer 2: Payload integrity — `LOGGER_HMAC_SECRET`

Every event body must carry an `hmac_signature` field. The signature is HMAC-SHA256 over the canonical string:

```
{request_id}|{client_timestamp_iso}|{event_type}|  
{sha256(canonical_payload_json)}
```

where `canonical_payload_json` is the event's `payload` dict serialised with `json.dumps(sort_keys=True, separators=(",", ":"), default=str)`.

The receiver re-derives the canonical string from the parsed body and compares (constant-time) against `hmac_signature`. Mismatch → HTTP 401.

This signature is **stored on the row** in the `hmac_signature` column. A downstream consumer or auditor can re-verify event integrity without trusting the original transport — they only need `LOGGER_HMAC_SECRET`.

Why two secrets instead of one

Compromise of `LOGGER_API_KEY` alone lets an attacker submit requests but not forge a valid signature — the integrity check still fails. Compromise of `LOGGER_HMAC_SECRET` alone is bad but doesn't grant network access if key rotation has already happened for the transport key. Rotating either secret does not invalidate the other.

Replay-window check

In addition to the HMAC, the receiver rejects events whose `client_timestamp` is more than `LOGGER_REPLAY_WINDOW_SECONDS` (default 300s) away from server time. This bounds the replay-attack window for any captured-and-resubmitted event and catches grossly skewed emitter clocks.

Canonical-string contract

The emitter and receiver **must** compute the same canonical string for the same logical event. The contract:

1. The payload dict is serialised with `json.dumps(sort_keys=True, separators=(",", ":"), default=str, ensure_ascii=False)`.
2. The resulting bytes are SHA-256 hashed.
3. The canonical string is built from `{request_id}|{iso_timestamp}|{event_type}|{payload_hash}`.
4. HMAC-SHA256 over the canonical string, using `LOGGER_HMAC_SECRET.encode("utf-8")` as the key.

Both `src/security.py` (here) and `src/client/logger.py` in `openagent-api` implement this. They must stay in lockstep.

Configuration reference

Every variable below has a default in code (or is documented as required). The full template is `.env.example` at the repo root.

Service identity / network (`src/api.py`)

Variable	Default	Description
<code>LOGGER_HOST</code>	<code>0.0.0.0</code>	Bind address inside the container
<code>LOGGER_PORT</code>	<code>8003</code>	Listen port; matches Dockerfile <code>EXPOSE</code>

Variable	Default	Description
LOGGER_ENABLE_DOCS	false	Expose Swagger at <code>/docs</code> (dev only)
LOGGER_RELOAD	false	Hot-reload on source changes (dev only)

Logging (src/api.py)

Variable	Default	Description
LOGGER_LOG_LEVEL	INFO	DEBUG, INFO, WARNING, ERROR, CRITICAL

Security (src/security.py)

Variable	Default	Description
LOGGER_API_KEY	(required)	X-API-Key transport secret
LOGGER_HMAC_SECRET	(required)	HMAC payload-signing secret
LOGGER_REPLAY_WINDOW_SECONDS	300	Max client_timestamp skew vs server

Generate strong values with:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Database (src/models.py)

Variable	Default	Description
LOGGER_DATABASE_URL	(empty)	Full URL; preferred. If set, components below are ignored.
LOGGER_DB_USER	openagent_logger	DB role
LOGGER_DB_PASSWORD	(required if no URL)	DB role password. Also passed to the Postgres container via <code>PGOPTIONS</code> at init time so the role is created with this exact value.
LOGGER_DB_HOST	openagent-shared-db	Hostname inside the Docker network
LOGGER_DB_PORT	5432	Port
LOGGER_DB_NAME	openagent_shared	Database name
LOGGER_DB_SCHEMA	openagent_logger	Schema inside that database
LOGGER_SQL_ECHO	false	Echo every SQL statement (debug only)

Retention (src/scheduler.py)

Variable	Default	Description
LOGGER_RETENTION_OPS_DAYS	90	Drop ops_events partitions older than this
LOGGER_RETENTION_CONVERSATION_DAYS	180	Same for conversation_captures
LOGGER_RETENTION_AUDIT_DAYS	2555	Same for audit_events (~7 years)
LOGGER_RETENTION_SCHEDULE_HOUR	3	UTC hour of day for the daily job

Local development setup

Prerequisites: shared infrastructure

1. Create the shared Docker network.

This network connects **openagent-logger** to the shared Postgres. It only needs to be created once per host machine. Both helper scripts are idempotent — safe to run any number of times.

```
# Windows / PowerShell
.\scripts\setup-network.ps1
```

```
# Linux / macOS
./scripts/setup-network.sh
```

2. Start the shared Postgres on that network. The **init.sql** from this repo is mounted into the container's init directory so the schema, tables, partitions, **and the fully-provisioned openagent_logger role** are created automatically on first boot.

The **openagent_logger** role's password is read from the custom GUC **logger.db_password**, which is set on the Postgres server via the **PGOPTIONS** environment variable.

```
# Linux / macOS
LOGGER_DB_PASSWORD="$(grep ^LOGGER_DB_PASSWORD .env | cut -d= -f2-)"

docker run -d --name openagent-shared-db \
  --network openagent-network \
  -e POSTGRES_PASSWORD='<admin secret>' \
  -e POSTGRES_DB=openagent_shared \
  -e PGOPTIONS="-c logger.db_password=${LOGGER_DB_PASSWORD}" \
  -v "$(pwd)/database/init.sql:/docker-entrypoint-initdb.d/openagent-logger-init.sql:ro" \
  -p 5432:5432 \
  postgres:16
```

Per-clone setup

```
# Copy the env template and fill it in
cp .env.example .env
$EDITOR .env

# Build and start
docker compose up --build

# Tail logs
docker compose logs -f openagent-logger

# Quick smoke test
LOGGER_API_KEY="$(grep ^LOGGER_API_KEY .env | cut -d= -f2)"
curl -fsS -H "X-API-Key: ${LOGGER_API_KEY}" http://localhost:8003/health
```

Local-only (no Docker) for tight iteration

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Make sure the shared Postgres is reachable from your host
export LOGGER_API_KEY="<dev key>"
export LOGGER_HMAC_SECRET="<dev secret>"
export LOGGER_DB_PASSWORD="<openagent_logger role pwd>"
export LOGGER_DB_HOST=localhost

python -m src.api
```

Resetting state during dev

```
# Wipe the logger schema (re-applies init.sql)
docker exec -i openagent-shared-db psql -U postgres -d openagent_shared \
    -c "DROP SCHEMA openagent_logger CASCADE;"
docker exec -i openagent-shared-db psql -U postgres -d openagent_shared \
    -f /docker-entrypoint-initdb.d/openagent-logger-init.sql
```

Deployment notes

Render

The same `Dockerfile` builds for Render. The Render dashboard is the source of truth for env vars (the `.env` file is local-only). Map `LOGGER_DATABASE_URL` to the Render Postgres add-on's connection URL; leave the `LOGGER_DB_*` component vars unset.

For the role-password handoff on Render, set `PGOPTIONS` on the admin/bootstrap shell that runs `init.sql`:

```
PGOPTIONS="-c logger.db_password=${LOGGER_DB_PASSWORD}" \  
psql "$DATABASE_URL" -f database/init.sql
```

A single Render web service runs one container instance. Because `openagent-logger` uses an **in-process APScheduler** for retention, do not scale to multiple instances without first solving the multi-fire problem (see Known limitations).

Self-hosted / docker-compose

The same `docker-compose.yml` works. Make sure `openagent-network` exists and the shared Postgres is attached to it (see Local development setup above).

Design decisions

Why HTTP + HMAC, not direct DB writes?

A service that reads from the same schema it writes to, on a hot path, is better off talking to the database directly — the extra HTTP hop is just latency. `openagent-logger` is the opposite case. Writes are append-only, never read on the hot path, and emitter and capture-layer are often on different hosts (gateway on a web tier, logger on a worker tier). HTTP+HMAC:

- Keeps the wire contract auditable independently of the DB schema.
- Lets me deploy `openagent-logger` on a different host, in a different network, behind a different firewall.
- Lets me scale capture independently of `openagent-api`.
- Stores the integrity signature on the row, so a downstream consumer can verify long after the original transport call is forgotten.

Why parallel to openagent-infra, not in series?

`openagent-api` could in principle proxy the conversation through `openagent-logger` on its way back to the frontend — making `openagent-logger` an inline observer. I chose parallel fan-out instead because:

- **Latency.** A series topology adds `openagent-logger`'s full INSERT to every `/chat` response. Parallel topology pushes that work off the hot path entirely.
- **Failure isolation.** When `openagent-logger` is down, `/chat` should keep working. In a series topology a logger outage breaks user-facing chat. In parallel it just drops capture events.

- **Reasoning chain hygiene.** The compute provider streams `delta.reasoning` and `delta.content` separately. By the time `openagent-api` calls `openagent-logger`, the assembled `output_text` is the user-visible answer only — the reasoning chain never reaches the capture table. A series topology would need explicit logic to strip it before forwarding.

Why APScheduler, not pg_cron?

`pg_cron` requires DB-side superuser configuration that not every managed Postgres exposes (notably Render). Keeping retention in application code:

- Survives DB upgrades and reprovisioning without manual cron re-setup.
- Is unit-testable with the same Python tooling as the rest of the service.
- Logs to the same stream as everything else (cross-service tailing).
- Lets me change retention behaviour without a DB migration.

Why three tables, not one polymorphic table?

A single `events` table with a JSONB `payload` column was considered. Three tables won because:

- Each event type has distinct retention. Mixing them on one partition table would require row-level filtering at retention time.
- Each event type has distinct indexes (`conversation_captures` needs `input_hash`; `audit_events` needs `actor`; `ops_events` needs `action`).
- The schemas are slightly different. A polymorphic table would either lose typing or accept the broadest superset of every field.

Why monthly partitions?

Daily partitions would create hundreds of partitions per table per year, which adds catalog overhead. Yearly partitions would blow past the retention granularity for `ops_events`. Monthly is the sweet spot: ~12 partitions per year per table, easy `DROP PARTITION` retention, manageable catalog size.

Why model-agnostic schema?

`conversation_captures.model_used` is `VARCHAR(255)`, not an enum. The schema doesn't hard-code any specific provider or model string: when the compute layer changes, the column accepts the new identifier without a migration. Downstream consumers filter by `model_used` at query time.

Why schema-scoped, not its own database?

The logger lives in its own schema (`openagent_logger`) inside the shared Postgres instance, with a role whose grants are scoped to that schema. That keeps the door open: another service can later share the same instance under its own schema and role without colliding with this one, and the logger can be lifted out to a dedicated Postgres at any time by changing only the connection URL — no code change. One instance is cheaper to run and back up than several, and schema-scoping buys the loose coupling without the operational cost.

Why `DROP PARTITION`, not `DELETE`?

Retention by `DELETE FROM ... WHERE created_at < cutoff` is:

- $O(\text{rows_to_delete})$ wall-clock time.
- Generates dead tuples that need VACUUM.
- Bloats indexes until VACUUM FULL.

Retention by `DROP TABLE <expired_partition>` is:

- $O(1)$ regardless of row count.
- Returns disk to the filesystem immediately.
- Leaves no dead tuples to clean up.

The trade-off is partition granularity — you can only retain in whole months, not days. For this use case that is the correct trade.

Why pass the role password via `PGOPTIONS`, not a separate `ALTER ROLE` step?

Passing the password as a custom GUC via `PGOPTIONS` lets `init.sql` read it during initial setup and create the role with the correct password in one transaction. This means the service connects on first boot rather than requiring a second manual debugging pass.

Known limitations

These are accepted-but-noted compromises. Each has a documented mitigation path if and when it matters.

Single-instance scheduler

The retention scheduler runs in-process via `APScheduler`. If you scale `openagent-logger` to multiple instances, every instance will run the daily retention job → multiple `DROP PARTITION` attempts (idempotent, but wasteful) and multiple `CREATE PARTITION IF NOT EXISTS` calls (also idempotent). It is functionally safe but not clean.

Mitigation: extract the scheduler into a dedicated sidecar, or use a distributed-lock pattern.

Rate limiting is upstream

`openagent-logger` has no per-emitter rate limit. A misconfigured emitter spamming events will be capped only by the database's connection pool.

Mitigation: a reverse proxy (Cloudflare, nginx) in front of the service with rate-limit rules.

HMAC canonical-string contract duplication

Both `src/security.py` and `src/client/logger.py` in `openagent-api` must compute the canonical string identically. There is no shared library yet.

Mitigation: extract to a shared package once both sides have stabilised.

Authenticated `/health` and `/stats`

These endpoints require **X-API-Key**. A naive load-balancer or platform healthcheck that does not know how to send the header will get 401 and mark the service unhealthy.

Mitigation: the in-container HEALTHCHECK in **docker/logger/Dockerfile** already sends the header **and** inspects the response body for **"status":"ok"** so it reflects the service's true state. Platform-level checks need to be configured to do the same, or to check **/**.

No transactional outbox on the emitter side

The emitter (**openagent-api**) is fire-and-forget. If **openagent-logger** is down when an event is emitted, the event is **lost** — there is no retry queue, no on-disk buffer.

Mitigation: accepted for now. **openagent-logger** is highly available in practice; permanent loss of an **ops_event** is operationally tolerable. For **conversation_captures** and **audit_events**, an outbox would be warranted.

File layout

```
openagent-logger/
├── src/
│   ├── __init__.py    package marker
│   ├── api.py         FastAPI app, lifespan, routes
│   ├── schemas.py     Pydantic envelope + per-type payloads
│   ├── models.py      SQLAlchemy models for the three tables
│   ├── security.py    X-API-Key + HMAC + replay window
│   ├── partitioning.py Create/drop partition helpers
│   └── scheduler.py   APScheduler daily retention job
├── database/
│   └── init.sql       Schema, parent tables, indexes, initial
partitions, grants
├── docker/
│   └── logger/
│       └── Dockerfile python:3.11-slim, user uid 1000, port 8003
├── scripts/
│   ├── setup-network.ps1 PowerShell: idempotent openagent-network
create
│   └── setup-network.sh  bash: idempotent openagent-network
create
├── docker-compose.yml  Local dev (single service, external
network)
├── requirements.txt    Python deps (pinned ranges)
├── .env.example        Env-var template (every var the code reads)
├── .dockerignore       Build-context exclusions (.env never in
image)
├── .gitignore          Standard Python + .env never in git
└── README.md           This file
```

License

Copyright © 2026 William McKeon.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

Maintainer

William McKeon (github.com/william-mckeon)